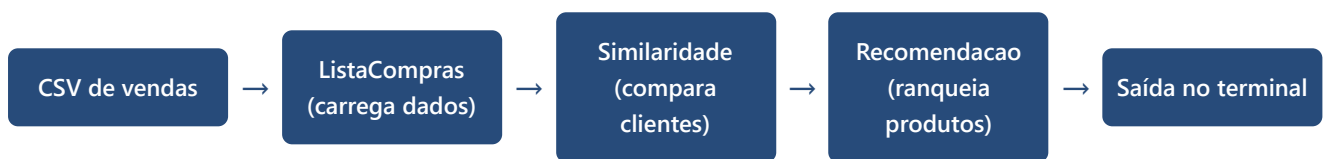


Fluxo de Funcionamento — Sistema de Recomendação

SistemaDeRecomendacao-PE-2026.1 • Explicação do código por módulo e função

1. Visão geral do algoritmo

O programa implementa um **sistema de recomendação por filtragem colaborativa item-a-item baseada em clientes**: ele lê um histórico de compras (CSV cliente/produto), mede o quão parecidos dois clientes são a partir dos produtos que compraram em comum, e usa os clientes mais parecidos ("vizinhos") de um cliente-alvo para sugerir produtos que ele ainda não comprou.



O ponto de entrada é `src/testador.cpp`, que escolhe um entre três modos de execução ("entregas") conforme o primeiro argumento de linha de comando:

entrega1

Só carrega o(s) CSV(s) e imprime estatísticas + as compras dos 3 primeiros clientes. Testa **ListaCompras**.

entrega2

Carrega e calcula a matriz de similaridade; mostra interseção/similaridade entre dois clientes e quem é o mais parecido com cada um. Testa **Similaridade**.

entrega3

Pipeline completo: carrega, calcula similaridade e gera o top-k de produtos recomendados para 3 clientes. Testa **Recomendacao**.

2. Módulo ListaCompras

include/ListaCompras.h · src/ListaCompras.cpp

Responsável por ler o CSV de vendas e transformá-lo em estruturas indexadas por inteiro (mais rápidas de manipular do que strings nos módulos seguintes).

Estrutura de dados

Campo	Tipo	Papel
<code>clientes</code>	<code>vector<string></code>	código do cliente na posição = índice interno
<code>mapa_clientes</code>	<code>map<string,int></code>	código → índice interno (lookup $O(\log n)$)
<code>produtos</code>	<code>vector<string></code>	nome do produto na posição = índice interno
<code>mapa_produtos</code>	<code>map<string,int></code>	código do produto → índice interno
<code>compras</code>	<code>vector<list<int>></code>	por cliente, lista dos índices de produtos comprados

lista_compras_carrega — leitura do CSV em duas passagens ListaCompras.cpp:13-60

Faz **duas passagens** pelo arquivo:

- **1ª passagem** (linhas 28-39): só descobre quais clientes e produtos existem, atribuindo a cada um um índice sequencial na ordem em que aparecem, e só então dimensiona `lc->compras` com `resize(clientes.size())`.
- **2ª passagem** (linhas 52-56): agora que os índices existem, percorre o arquivo de novo e empilha, para cada linha, o índice do produto na lista de compras do respectivo cliente.

Por que duas passagens?

Como `compras` é indexado pela posição do cliente e essa posição só é conhecida depois de ver todo o arquivo pelo menos uma vez, a forma mais simples de garantir os índices antes de preencher as listas é ler o arquivo duas vezes, em vez de usar estruturas dinâmicas mais complexas.

```

bool lista_compras_carrega(ListaCompras *lc, const char *caminho_arquivo) {
    ...
    // 1a passagem: apenas registra clientes/produtos novos e cria seus indices
    while (fscanf(arquivo, "%*[^,],%31[^,],%15[^,],%63[^\\n]\\n", cod_cliente, cod_produto,
nome_produto) == 3) {
        if (lc->mapa_clientes.find(cod_cliente) == lc->mapa_clientes.end()) {
            int indice = lc->clientes.size();
            lc->clientes.push_back(cod_cliente);
            lc->mapa_clientes[cod_cliente] = indice;
        }
        if (lc->mapa_produtos.find(cod_produto) == lc->mapa_produtos.end()) {
            int indice = lc->produtos.size();
            lc->produtos.push_back(nome_produto);
            lc->mapa_produtos[cod_produto] = indice;
        }
    }
    lc->compras.resize(lc->clientes.size());
    ...
    // 2a passagem: preenche a lista de compras de cada cliente com os indices de produto
    while (fscanf(arquivo, "%*[^,],%31[^,],%15[^,],%63[^\\n]\\n", cod_cliente, cod_produto,
nome_produto) == 3) {
        int ic = lc->mapa_clientes[cod_cliente];
        int ip = lc->mapa_produtos[cod_produto];
        lc->compras[ic].push_back(ip);
    }
    ...
}

```

`%*[^,]` descarta a 1ª coluna do CSV (ex.: id da linha/data), e as três colunas seguintes são capturadas como cliente, código do produto e nome do produto.

Funções auxiliares

- `lista_compras_indice_cliente` / `lista_compras_indice_produto` (linhas 62-72): traduzem um código de cliente/produto para o índice interno usando os mapas; retornam `-1` se não encontrado.
- `lista_compras_imprime_compras` (linhas 74-83): imprime o nome de cada produto comprado por um cliente — usada pela `entrega1`.

3. Módulo Similaridade

include/Similaridade.h · src/Similaridade.cpp

Transforma a lista de compras em matrizes e calcula, para cada par de clientes, o quão parecidos eles são, usando álgebra de matrizes sobre uma matriz binária de compras.

Estrutura de dados

Campo	Papel
<code>matriz_compras</code>	matriz binária $n \times m$: 1 se o cliente i comprou o produto j
<code>matriz_intersecao</code>	matriz $n \times n$: nº de produtos em comum entre cada par de clientes
<code>matriz_similaridade</code>	matriz $n \times n$ de "distância" entre 0 (idêntico) e 1 (nada em comum)

`similaridade_monta_matriz_compras` `Similaridade.cpp:23-38`

Percorre `lc->compras` e marca com `1` a célula `[cliente][produto]` sempre que houve a compra — construindo a matriz binária clientes $\times$ produtos que alimenta todo o resto do módulo.

`similaridade_transposta e similaridade_multiplica_matrizes` `Similaridade.cpp:40-63`

São utilitários genéricos de álgebra linear com matrizes alocadas dinamicamente (`malloc` / `calloc`): transposição de uma matriz e multiplicação de matrizes (produto usual linha $\times$ coluna, $O(n \cdot m \cdot n)$).

`similaridade_calcula` — o núcleo do módulo `Similaridade.cpp:65-85`

Ideia-chave: interseção via multiplicação de matrizes

Multiplicando a matriz de compras ($n \times m$) pela sua transposta ($m \times n$) obtém-se uma matriz $n \times n$ em que a célula `[i][j]` é exatamente o **número de produtos comprados tanto pelo cliente i quanto pelo cliente j** — e a diagonal `[i][i]` dá o total de produtos distintos comprados pelo próprio cliente i .

```
void similaridade_calcula(Similaridade *sim, const ListaCompras *lc) {
    similaridade_monta_matriz_compras(sim, lc);

    Matriz transposta = similaridade_transposta(sim->matriz_compras, sim->n, sim->m);
    // intersecao = matriz_compras (n x m) * transposta (m x n) => n x n
    sim->matriz_intersecao = similaridade_multiplica_matrizes(sim->matriz_compras, sim->n, sim->m,
transposta, sim->n);
    ...
    for (int i = 0; i < sim->n; i++) {
        int total_produtos_i = sim->matriz_intersecao[i][i]; // diagonal = total do cliente i
        for (int j = 0; j < sim->n; j++) {
            sim->matriz_similaridade[i][j] = 1.0 - (double) sim->matriz_intersecao[i][j] /
total_produtos_i;
        }
    }
}
```

Atenção: apesar do nome, é uma medida de distância

`matriz_similaridade[i][j] = 1 - interseção/total_i` . Quanto **menor** o valor, **mais parecidos** são os clientes (0 = compraram exatamente o mesmo conjunto de produtos que o cliente i já tem; próximo de 1 = quase nada em comum). Essa convenção "menor = mais similar" é usada por todo o resto do sistema, inclusive no módulo de recomendação.

similaridade_mais_similar `Similaridade.cpp:87-96`

Varre a linha do cliente na matriz de similaridade (ignorando a própria posição) e retorna o índice do cliente com o **menor** valor — ou seja, o vizinho mais parecido, segundo a convenção acima.

4. Módulo Recomendacao

include/Recomendacao.h · src/Recomendacao.cpp

Usa a matriz de similaridade para gerar uma lista ranqueada de produtos que um cliente ainda não comprou, priorizando produtos comprados por clientes parecidos com ele.

recomendacao_vizinhos Recomendacao.cpp:5-16

Considera "vizinho" qualquer outro cliente cuja similaridade (distância) com o cliente-alvo seja `< 1.0`, isto é, qualquer cliente que tenha ao menos um produto em comum com ele.

recomendacao_calcula_ranking — como o score é montado Recomendacao.cpp:18-40

```
vector recomendacao_calcula_ranking(...) {
    ...
    for (int p = 0; p < m; p++) r[p].ranqueamento = 1.0;    // todo produto começa com score 1.0

    int *vizinhos = recomendacao_vizinhos(sim, indice_cliente, &total_vizinhos);
    for (int i = 0; i < total_vizinhos; i++) {
        int s = vizinhos[i];
        for (int p = 0; p < m; p++) {
            bool s_comprou = sim->matriz_compras[s][p] == 1;
            bool c_nao_comprou = sim->matriz_compras[indice_cliente][p] == 0;
            if (s_comprou && c_nao_comprou) {
                // multiplica pelo valor de "distancia" do vizinho: quanto mais parecido, mais baixa
                // a distancia
                r[p].ranqueamento *= sim->matriz_similaridade[indice_cliente][s];
            }
        }
    }
    ...
}
```

Como interpretar o score

Cada produto começa com `ranqueamento = 1.0`. Para cada vizinho que comprou aquele produto (e que o cliente-alvo ainda não comprou), o score é multiplicado pela distância do vizinho. Como essa distância é sempre `< 1.0`, o score só diminui — e diminui mais quanto mais vizinhos parecidos compraram o produto e quanto mais parecidos eles são. No final, **menor ranqueamento = recomendação mais forte**.

recomendacao_compara_ranking e recomendacao_top_k Recomendacao.cpp:42-52

Ordena todos os produtos em ordem **crescente** de `ranqueamento` (desempate pelo índice do produto) e devolve os `k` primeiros — ou seja, os k produtos com o menor (melhor) score.

5. testador.cpp — orquestração e ponto de entrada

`main()` valida os argumentos, escolhe a "entrega" pedida e monta o pipeline correspondente chamando os três módulos acima na ordem certa.

entrega1 — só ListaCompras `testador.cpp:25-42`

Para cada CSV informado (ou o CSV padrão `data/dados_venda_cluster_17.csv` se nenhum for passado): chama `lista_compras_carrega`, imprime nº de clientes/produtos e as compras dos 3 primeiros clientes via `lista_compras_imprime_compras`.

entrega2 — ListaCompras + Similaridade `testador.cpp:44-78`

Carrega o CSV, chama `similaridade_calcula` e, para os dois índices de cliente informados (padrão 0 e 1), imprime: total de produtos do cliente, interseção e similaridade entre o par, e o resultado de `similaridade_mais_similar` para cada um.

entrega3 — pipeline completo `testador.cpp:80-114`

```
ListaCompras lc; inicializa_lista_compras(&lc); lista_compras_carrega(&lc, caminho);
Similaridade sim; inicializa_similaridade(&sim); similaridade_calcula(&sim, &lc);

for (int i = 0; i < 3; i++) {
    int indice = lista_compras_indice_cliente(&lc, codigos[i]);
    vector topk = recomendacao_top_k(&sim, &lc, indice, k); // ListaCompras + Similaridade ->
    Recomendacao
    for (int j = 0; j < (int) topk.size(); j++)
        cout << lc.produtos[topk[j].indice_produto] << " (Rank=" << topk[j].ranqueamento << ")" <<
endl;
}
similaridade_libera(&sim);
```

Este é o caminho que amarra os três módulos: `ListaCompras` fornece os dados brutos, `Similaridade` calcula o quão parecidos os clientes são, e `Recomendacao` usa essa comparação para ranquear e devolver os `k` melhores produtos para cada um dos 3 clientes informados (ou os 3 primeiros do arquivo, por padrão). Ao final, `similaridade_libera` desaloca as matrizes criadas com `malloc` / `calloc`.

Gerado a partir do código-fonte em `src/` e `include/` do projeto SistemaDeRecomendacao-PE-2026.1.